Daffodil International University
Department of Software Engineering

System Design Document

Unit 5: Game-Account Escrow

TrustBazaar: A Verified Peer-to-Peer Auction and Marketplace Platform for Used Goods

S. M. Nihal Ahmed (232-35-002)
Afrim Hossen Khan (232-35-076)
Sabikun Nahar Sinthia (232-35-475)

Status	Draft for team review
Version	0.1, 18 October 2026
Implements	SRS v2.0 (`docs/srs.tex`)
Depends on	Unit 1 (Identity & Access), Unit 2 (Order, Payment & Escrow), Unit 3 (Auctions, Listings & Search); Unit 4 not used
Database schema	`docs/design/sql/05_game_accounts.sql`, tested on PostgreSQL (70 checks, concurrency tests) and the browser code in Chromium (14 checks); Appendix B

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Requirements covered . . . 3

2 Unit Overview **4**
2.1 Responsibilities . . . 4
2.2 Who can see what . . . 4
2.3 Components . . . 4

3 Interfaces **5**
3.1 HTTP API . . . 5
3.2 Ports provided . . . 6
3.3 Events . . . 6
3.4 Ports required . . . 6

4 Data Design **7**
4.1 Tables and rules the database enforces . . . 7

5 Listing a Game Account **8**
5.1 Details . . . 8
5.2 Ownership proof . . . 8
5.3 One live listing per player ID . . . 8

6 The Sealed Handover **9**
6.1 Choice of construction . . . 9
6.2 The buyer's key . . . 9
6.3 Fingerprint . . . 9
6.4 Sealing and delivery . . . 10
6.5 Retrieval and the access log . . . 11
6.6 Re-key . . . 11

7 Takeover and Protection **11**
7.1 The checklist . . . 11
7.2 Completion . . . 11
7.3 "The credentials do not work" . . . 12
7.4 Protection period and seller recovery . . . 12
7.5 Deletion . . . 13

8 Scheduled Jobs **13**

9 Security **13**

9.1 Controls 13

9.2 Threats considered 13

10 Parameters **14**

11 Testing **15**

12 Notes for Integration **15**

A Traceability **16**

B Database Schema and Tests **17**

B.1 Test results 17

B.2 Schema 19

C Browser Module **28**

1 Introduction

1.1 Purpose

This document is the fifth unit of TrustBazaar's System Design Document. It covers what a game account needs in place of physical delivery:

- **proof of ownership** before the listing is published;
- **a sealed credential handover**: the seller's browser encrypts the login details with a key that only the buyer's browser holds, so the platform stores them but can never read them;
- **a guided takeover**, whose completion is the order's delivery event;
- **a protection period**, during which the payment stays in escrow in case the seller recovers the account.

The unit's central claim is security objective 2 of the SRS: *the server and its staff never hold game-account credentials in readable form* (SEC-GAM-01). This document shows how that is achieved and tested, and states plainly where the protection stops (SRS assumption AS-6).

The unit follows conventions DC-1 to DC-14 (Units 1, 2 and 4). It uses Unit 2's escrow through `EscrowCommands`, approves listings through Unit 3's `ListingsCommands`, and does not use Unit 4: game accounts have no delivery (FR-GAM-06).

1.2 Scope

In scope:

- the games catalogue and the structured game fields of a listing;
- ownership codes, proof screenshots, their review, and one live listing per player ID;
- the publisher-risk disclosure and its acknowledgement;
- the buyer's key pair, the fingerprint, the sealing and opening of credentials in the browser, and the credential-access log;
- the takeover checklist, the takeover window, re-keying, `credentials_invalid` reports, and deletion of the ciphertext;
- the evidence needed when a seller recovers a sold account (FR-GAM-18);
- the feature switch CP-36.

Out of scope, owned by other units:

- the protection period's escrow hold, early release and the payout (Unit 2, release policy `game_protection`);
- the listing itself, its price, photos and visibility (Unit 3);
- notifications and the chat warning of FR-GAM-20 (Unit 6);
- disputes, the ownership-proof queue screens, suspension, blacklisting and the evidence export (Unit 7, with Unit 1's approvals);
- the screens themselves (Unit 8); this document fixes what they must do and supplies the browser code they use.

1.3 Requirements covered

- **Functional:** FR-GAM-01 to FR-GAM-20; FR-PAY-07 and FR-PAY-08 (game side); FR-DSP-01 (game windows); FR-DSP-05 (game evidence); FR-NTF-07 and FR-NTF-08 (deadlines; nothing secret in notifications).
- **Security:** SEC-GAM-01 to SEC-GAM-03; SEC-DAT-07, SEC-DAT-10; SEC-AUTH-08 (step-up for early release, applied by Unit 2); SEC-IRM-04 (switching the feature off).

- **Other:** IF-UI-06, IF-SW-07; NFR-USE-03; NFR-MNT-02 (game-account release rules); DR-07; AS-5, AS-6.

2 Unit Overview

2.1 Responsibilities

1. Publish a game-account listing only after an operator has checked a fresh ownership code on the seller's game profile, and never let two live listings claim the same player ID.
2. Get the login details from the seller's browser to the buyer's browser without the platform ever being able to read them.
3. Record exactly one fulfilment step (first credential delivery) and one delivery event (takeover complete) for Unit 2's escrow, at the right moments.
4. Keep a complete, append-only record of who retrieved the sealed credentials and which takeover steps the buyer confirmed, for disputes and, if needed, for a complaint to law enforcement.
5. Delete the ciphertext as soon as the order is over.

2.2 Who can see what

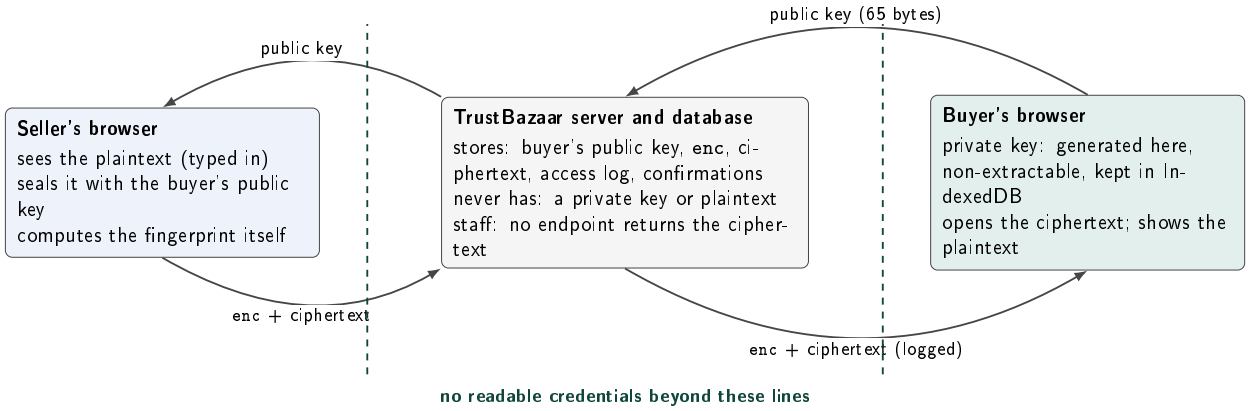


Figure 1: Trust boundary of the sealed handover. The server relays and stores only what it cannot decrypt (SEC-GAM-01). AS-6: this assumes the code the server sends to the browsers has not been altered.

2.3 Components

Component	Responsibility
Backend module `game-accounts`	
GameListingService	Game fields; issues ownership codes; receives proof screenshots; refuses listings while CP-36 is off (Section 5).
OwnershipReviewService	Operator review of proofs; takes the player-ID claim; asks Unit 3 to publish (Section 5.2).
HandoverService	Disclosure, buyer key, credential delivery and retrieval, re-key, reports; the DC-14 calls to `EscrowCommands` (Section 6).
TakeoverService	Checklist confirmations and completion (Section 7).
GameJobs	Window-end completion, re-key overdue, proof expiry (Section 8).
EvidenceService	Assembles the evidence for disputes and for FR-GAM-18.

Component	Responsibility
Browser module `sealed.js` (Appendix C), used by the Unit 8 screens	
Key manager	Generates the buyer's non-extractable key pair and keeps it in IndexedDB.
Sealer / opener	HPKE seal in the seller's browser, open in the buyer's; padding; the fingerprint.

3 Interfaces

3.1 HTTP API

All paths are under `/api/v1`; policies and rate limits are those of Unit 1. Binary values travel as base64url. Responses that carry a public key or a ciphertext are sent with `Cache-Control: no-store`, and no response ever carries a fingerprint (Section 6.3).

Method	Path	Policy	Purpose (requirement)
Listing a game account			
`GET`	`/games`	Public	Games, platforms, where the code goes, the publisher note
`PUT`	`/listings/{id}/game-details`	Verified(list), seller	Game, platform, player ID, contents (FR-GAM-02)
`POST`	`/listings/{id}/ownership-code`	Seller	Issue a code valid for DV-33 (FR-GAM-03)
`POST`	`/listings/{id}/ownership-proof`	Seller	Upload the screenshot; submits the proof
Checkout (before payment)			
`POST`	`/orders/{id}/game/acknowledge`	Buyer	Accept the disclosure, by version (FR-GAM-05)
`POST`	`/orders/{id}/game/key`	Buyer	Register the browser's public key (FR-GAM-07)
Handover and takeover (after payment)			
`GET`	`/orders/{id}/game`	Buyer or seller	Status, deadlines, countdowns, confirmed steps (NFR-USE-03)
`GET`	`/orders/{id}/game/buyer-key`	Seller	The buyer's current public key and its version
`POST`	`/orders/{id}/game/sealed`	Seller	Deliver `enc` and ciphertext for that version (FR-GAM-09, 10)
`GET`	`/orders/{id}/game/sealed`	Buyer	Retrieve them; every call is logged (FR-GAM-11)
`POST`	`/orders/{id}/game/takeover-steps/{step}`	Buyer	Confirm a checklist step (FR-GAM-13)
`POST`	`/orders/{id}/game/report-invalid`	Buyer	"The credentials do not work": opens a dispute (FR-GAM-14)
`POST`	`/orders/{id}/game/rekey`	Buyer	New public key after losing the old one (FR-GAM-12)
`POST`	`/orders/{id}/release-early`	Buyer, StepUp	Unit 2's endpoint (FR-GAM-17, SEC-AUTH-08)
Staff			
`GET`	`/staff/game/ownership-proofs`	Staff(operator)	The proof queue (FR-OPS-01)
`GET`	`/staff/game/ownership-proofs/{id}/screenshot`	Staff(operator)	Signed link valid CP-31; each view audited
`POST`	`/staff/game/ownership-proofs/{id}/review`	Staff(operator)	Approve, or reject with a reason, recording the live-profile check

Method	Path	Policy	Purpose (requirement)
`POST`, `PATCH`	`/admin/games`	Staff(admin)	Maintain the games catalogue

There is deliberately no staff endpoint that returns a ciphertext or a public key, and the evidence for a case (Section 7.4) contains neither. Refusals return `422` with a stable code, among them `feature_off`, `player_id_claimed`, `code_expired`, `key_fixed`, `stale_key_version`, `deadline_passed`, `window_ended`, `recovery_email_confirmed` and `rekey_used`.

3.2 Ports provided

```
interface GameAccountPort {                          // used by Unit 2
  assertCheckoutReady(orderId: string): Promise<void>;   // disclosure acknowledged, key registered, CP-36 on
}
interface GameAccountCommands {                      // used by Unit 7
  reviewProof(proofId: string, staffId: string, approve: boolean,
              liveCheck: 'matched' | 'not_matched' | 'not_available', reason?: string): Promise<void>;
  disputeWithdrawn(orderId: string, tx: EntityManager): Promise<void>;
}
interface GameAccountQueries {                       // used by Units 6 and 7
  evidence(orderId: string): Promise<GameEvidence>;           // FR-DSP-05, FR-GAM-18
  credentialsInvalidAllowed(orderId: string): Promise<boolean>;  // FR-GAM-14
  deadlines(orderId: string): Promise<GameDeadlines>;         // for reminders (FR-NTF-07)
}
```

3.3 Events

Event	Use
Published (outbox, DC-6). No payload ever contains key material or ciphertext.	
`game_accounts.ownership.approved`, `.rejected`	Unit 3 publishes or returns the listing; notifications.
`game_accounts.credentials.requested`	Notifications to the seller, with the CP-22 deadline.
`game_accounts.credentials.delivered`	Notifications to the buyer, with the takeover deadline.
`game_accounts.credentials.reported_invalid`	Disputes (opened in the same transaction); notifications.
`game_accounts.rekey.requested`, `.overdue`	Notifications to the seller; Disputes when overdue.
`game_accounts.takeover.completed`	Notifications: the protection period's end date. The escrow change is made in the same transaction (DC-14).
Consumed (idempotent)	
`orders.order.paid` (game account)	`mark_paid`: ask the seller for the credentials, with the order's CP-22 deadline (FR-GAM-10).
`orders.escrow.refunded` before the delivery event	`finish(before)`: handover cancelled, ciphertext deleted.
`orders.escrow.released`, `.refunded`, `.partially_refunded` after it	`finish(after)`: handover closed, ciphertext deleted, player-ID claim released (FR-GAM-19, FR-GAM-04).
`listings.listing.ended_unsold`, `.cancelled`, `.removed`	Release the claim of an unsold listing.

3.4 Ports required

Port	Provider and use
`IdentityQuery`, `AuditService`, `FileVault` (Unit 1)	`assertCanTransact`; audit of reviews and screenshot views; encrypted storage of proof screenshots under a new purpose (I-31).
`OrdersPort`, `EscrowCommands` (Unit 2)	`lockForFulfilment` (DC-14); `recordFulfilmentStep(credentials_delivered)`; `recordDeliveryEvent(game_takeover, at)`.
`ListingsCommands` (Unit 3)	`approveForPublication` and `returnToDraft` after a review.
`DisputeCommands` (Unit 7)	Open a `credentials_invalid` dispute in the same transaction as the report.
`ConfigPort`, `Clock`	CP-22, CP-23, CP-24, CP-36 and the DV values.

4 Data Design

The schema is in `docs/design/sql/05_game_accounts.sql` (Appendix B), loaded after Units 1 to 4. The unit owns the schema `game_accounts`. Its foreign keys point to users, staff and sessions (Unit 1), orders (Unit 2) and listings (Unit 3), as DC-5 allows.

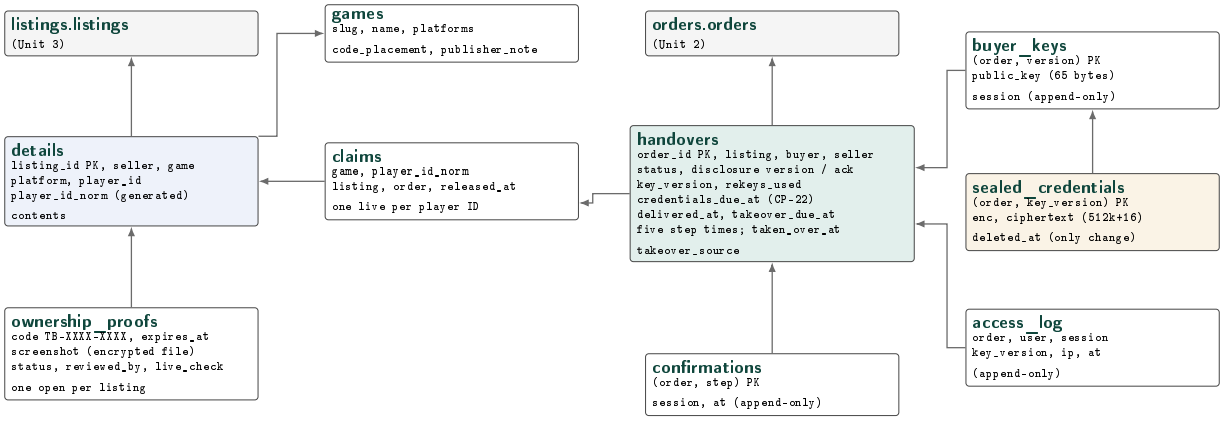


Figure 2: The `game_accounts` schema. Handovers also reference the listing's `details`; claims reference the order once sold. Those arrows are omitted.

4.1 Tables and rules the database enforces

Table	Rules enforced by the database
`games`	Slug format; at least one platform.
`details`	The platform must be one the game offers. Player ID printable, at least 2 characters, compared without spaces or case. Game and player ID fixed once a proof is submitted.
`ownership_proofs`	Code format (DV-33), unique. One open proof per listing. Only the permitted status changes. Submitting needs the screenshot. A review records the operator, the time and the live-profile check; a rejection needs a reason; approval is impossible if the live profile did not match, or after the code has expired. A reviewed proof is final.
`claims`	One live claim per game and player ID (FR-GAM-04), taken only by an approval.

Table	Rules enforced by the database
`handovers`	Starts as `preparing` without a key. Parties and disclosure fixed. Only the permitted status changes; cancelled and closed are final. A paid handover has a key and the CP-22 deadline. Status fields consistent. Key version = re-keys + 1, at most one re-key. Confirmed steps and the takeover time can never change. A checklist completion needs all five steps. A cancellation is impossible after the takeover. A closed or cancelled handover can have no ciphertext left (trigger).
`buyer_keys`	Uncompressed P-256 point (65 bytes, `0x04`); append-only.
`sealed_credentials`	For an existing key version. `enc` is a P-256 point; ciphertext is $512k + 16$ bytes, at most 4 KB of plaintext. The only change ever allowed is deletion of the content.
`access_log`, `confirmations`	Append-only; one confirmation per step.

5 Listing a Game Account

5.1 Details

The seller creates the listing in Unit 3 as usual (title, description, price, fixed price or auction) and adds the game fields here: the game from the catalogue, the platform, the in-game player ID and a description of the account's contents (FR-GAM-02). The form never asks for any login detail, and Unit 3's screening terms include common words for passwords in Bangla and English, so a seller who types one into the description is held for review. While CP-36 is off, game-account listings and purchases are refused, and active ones are hidden; paid orders continue (FR-GAM-01). The switch is changed through Unit 1's administrator controls with four-eyes approval (SEC-IRM-04).

5.2 Ownership proof

1. The seller asks for a code. It looks like `TB-7K4Q-9XWM`: 40 random bits in Crockford base32, without the easily confused letters I, L, O and U, valid for DV-33 (7 days). The code is not a secret; it proves that the seller controls the profile *now*.
2. The seller puts the code where the catalogue says (for example the profile signature) and uploads a screenshot showing the code and the player ID. The screenshot is Confidential, so it goes through Unit 1's `FileVault`: re-encoded without metadata, encrypted with its own data key, stored on a private volume, and shown to operators only through a signed link valid for CP-31 (each view audited).
3. An operator compares the screenshot with the listing and, where the game has public profiles, opens the live profile and looks for the code. The result is recorded as `matched`, `not_matched` or `not_available`. A screenshot alone can be faked, so a mismatch on the live profile makes approval impossible (database constraint).
4. Approval takes the player-ID claim and asks Unit 3 to publish. Rejection needs a reason, which the seller sees.

5.3 One live listing per player ID

FR-GAM-04 allows at most one active listing per game and player ID, and none while an earlier sale is still in its takeover window or protection period. Both are met by one rule: a *claim* on the player ID, unique while live.

- The claim is taken *when ownership is approved*, not when a draft is written. Otherwise anyone could block another person's player ID by drafting a listing for it.

- It is released when the listing ends unsold, is cancelled or is removed, or, after a sale, when the order's escrow reaches a terminal state, which for a game account is after the protection period.
- Player IDs are compared without spaces and case, so `5123 4567` and `51234567` are the same.

Tested: ten operators approving ten listings of the same player ID at the same moment produced exactly one approval and one live claim.

6 The Sealed Handover

6.1 Choice of construction

SEC-DAT-10 allows two options: RSA-OAEP with SHA-256 and a key of at least 3072 bits, or ECDH on P-256 with AES-256-GCM. RSA-OAEP can encrypt at most $384 - 2 \times 32 - 2 = 318$ bytes with a 3072-bit key, which a login, a long password and a set of recovery codes can exceed. The design therefore uses the ECDH option, in its standard form: **HPKE** (Hybrid Public Key Encryption, RFC 9180) in base mode, with the suite DHKEM(P-256, HKDF-SHA256), HKDF-SHA256 and AES-256-GCM (DV-35).

HPKE is the IETF's standard way of combining ECDH, a key-derivation function and AES-GCM. Using it means no construction of our own (C-4). It is implemented by the library `@hpke/core` (version 1.9.0), which is built only on the browser's Web Cryptography API (IF-SW-07). The team's own code is the thin module `sealed.js` in Appendix C. Each sealing uses a fresh ephemeral key, whose public half is sent as `enc` with the ciphertext. Two values bind a ciphertext to its context:

- the HPKE *info* string `TrustBazaar/game-credentials/v1`;
- the AES-GCM *associated data* `order:<order id>|key:<key version>`, so a ciphertext copied to another order, or replayed under another key version, fails to open (tested).

6.2 The buyer's key

At checkout, after accepting the disclosure (FR-GAM-05), the buyer's browser:

1. checks that the Web Cryptography API and IndexedDB are available, and otherwise explains that this browser cannot take part in a game-account sale (AS-5);
2. generates a P-256 key pair with `extractable = false` (FR-GAM-07). The private key can be used for decryption, but the browser refuses to export it in any format (tested: `pkcs8` and `jwk` export both fail);
3. stores the key pair in this site's IndexedDB under the order id and key version. IndexedDB keeps the key as a `CryptoKey` object, still non-extractable after a reload (tested);
4. sends only the 65-byte public key, which the server stores for the order;
5. tells the buyer to finish the purchase and the takeover in this browser, on this device.

Unit 2 accepts payment only when `assertCheckoutReady` confirms the disclosure acknowledgement and the key. Before payment the key cannot be replaced; after payment only the one re-key of Section 6.6 can replace it.

6.3 Fingerprint

Both browsers show a short fingerprint of the buyer's public key: the first 80 bits of its SHA-256, as five groups of four hex digits, for example `cce1-9bdb-3a7c-0768-1d4f` (DV-36, FR-GAM-08). The buyer's browser computes it from the key it generated, and the seller's from the key it received. If the two match when read out, the server did not swap the key.

The fingerprint is worth exactly as much as the channel used to compare it. Compared over TrustBazaar's own chat, it detects a key changed in the database, but not a fully compromised server that could also change the chat (AS-6). The screens therefore suggest a phone call for high-value sales. For the same reason the server never sends a fingerprint: one computed by the server would prove nothing. The database function `game_accounts.fingerprint` exists only as the reference for tests, and the browser and database results for the same test vector are compared (both `0ed3-a6ab-957f-f6f5-9a96`).

6.4 Sealing and delivery

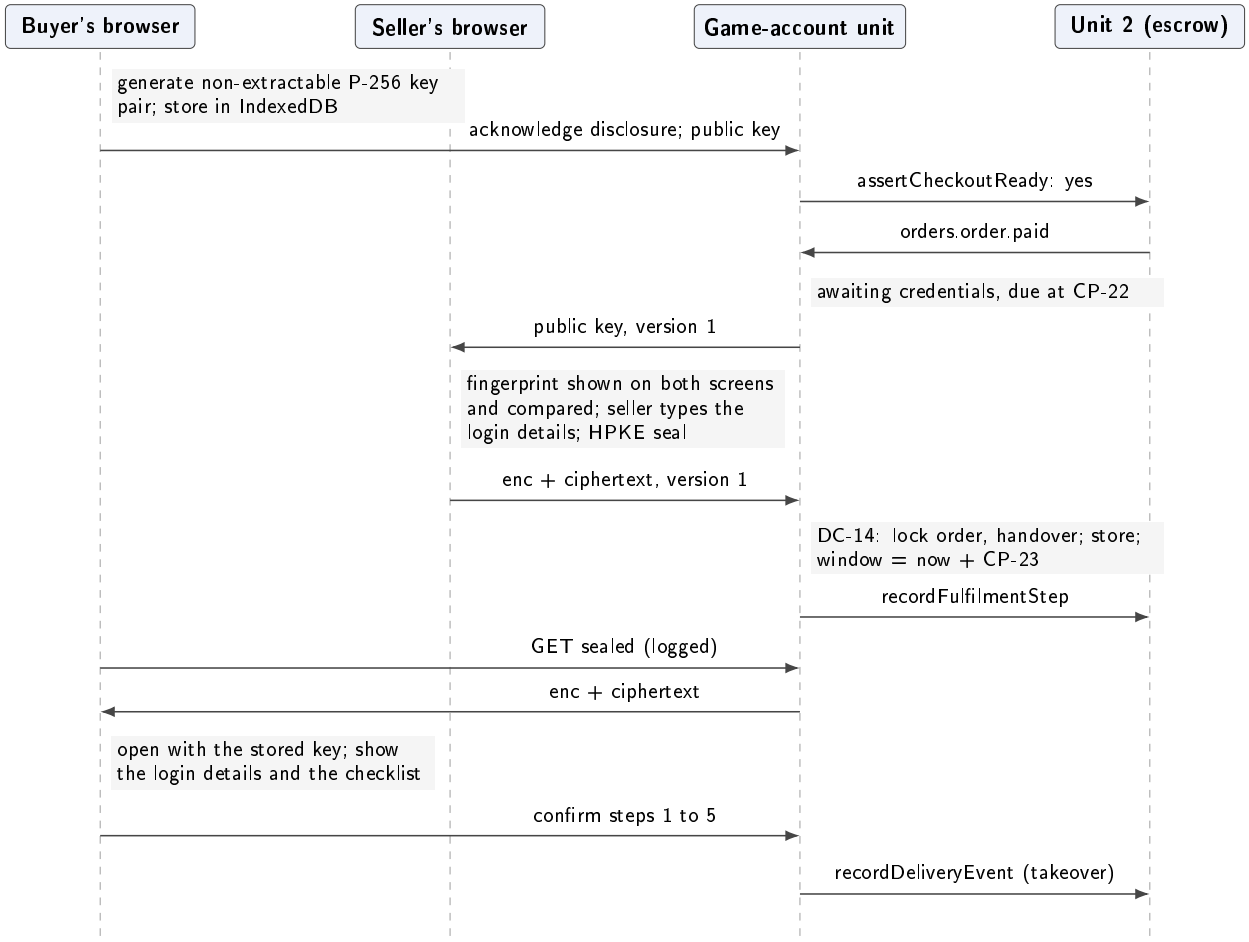


Figure 3: Sealed handover and takeover. Nothing to the right of the browsers can read the login details.

- **When.** The seller is asked for the credentials only after payment is `held`, and must deliver them by CP-22 (24 hours after payment); otherwise Unit 2 refunds the buyer in full (FR-GAM-10). The database refuses a delivery at or after that deadline.
- **What.** The seller's screen has fields for the login identifier, the password, recovery codes and a note. The fields have spell-checking and autocomplete turned off, because some browsers send spell-checked text to an online service. The values are serialised to JSON, prefixed with their length and padded with zeros to a multiple of 512 bytes, at most 4 KB (DV-34). Padding hides the password's length: a 5-character and a 300-character password give ciphertexts of the same size (tested).
- **For which key.** The ciphertext names the key version it was sealed for. The database refuses a version that is not the buyer's current one, so a seller who sealed for the old key after a re-key is told to seal again.

- **Exactly once.** A second delivery for the same key is refused. Ten simultaneous deliveries produced one ciphertext. The first delivery is reported to Unit 2 as the fulfilment step.

6.5 Retrieval and the access log

Only the buyer of the order, from an authenticated session, can retrieve the ciphertext (FR-GAM-11). Every retrieval writes a row to the append-only access log: order, user, session, key version, source address and time (SEC-GAM-02). Twenty simultaneous retrievals produced twenty log rows. The log is kept for 2 years after the order closes (SRS retention schedule). The opened credentials exist only in the page: the view hides them after DV-38 (2 minutes) without activity, and the copy button clears the clipboard after 60 seconds where the browser allows.

6.6 Re-key

A buyer who loses the private key, for example by clearing browser data or changing device, can ask for one re-key during the takeover window (FR-GAM-12):

1. the new browser generates a new key pair (version 2) and sends its public key;
2. the takeover window stops; the old ciphertext stays readable but is useless to the buyer;
3. the seller is asked to seal again for version 2, by DV-37 (CP-22, 24 hours, after the request);
4. the new delivery deletes the old ciphertext and starts a new full takeover window. It is not a second fulfilment step for Unit 2.

A re-key is refused once the buyer has confirmed changing the recovery e-mail: by then the buyer controls the account and needs no credentials. Only one re-key is allowed; ten simultaneous requests produced one new key. If the seller does not seal again in time, the handover becomes `disputed`, and Unit 7 opens a `credentials_invalid` dispute on the buyer's behalf, because the buyer can no longer get in through no fault of the platform's rules.

7 Takeover and Protection

7.1 The checklist

When the buyer opens the credentials, the screen shows the takeover checklist with a countdown of the window and the date on which the protection period will end (FR-GAM-13, NFR-USE-03):

1. change the password;
2. change the recovery e-mail;
3. change the recovery phone;
4. remove linked social logins;
5. turn on two-step verification.

Each step is confirmed by the buyer and stored once, with its session and time, in the append-only `confirmations` table. A confirmed step can never be undone.

7.2 Completion

The takeover is complete, and the order's delivery event happens, at the first of:

- the buyer's confirmation of the fifth step; ten simultaneous confirmations of the last step produced one completion and one event;
- the end of the takeover window CP-23 (24 hours after delivery), if the buyer has neither finished nor reported a problem (FR-GAM-15). The completion time recorded is the end of

the window, not the time the job happened to run, so the protection period is not lengthened by job delay.

On completion the service calls `recordDeliveryEvent(game_takeover, at)` in the same DC-14 transaction. Unit 2 then holds the payment until `at` + CP-24 (7 days), under its `game_protection` release policy, and the general auto-release CP-19 does not apply (FR-GAM-16, FR-GAM-17).

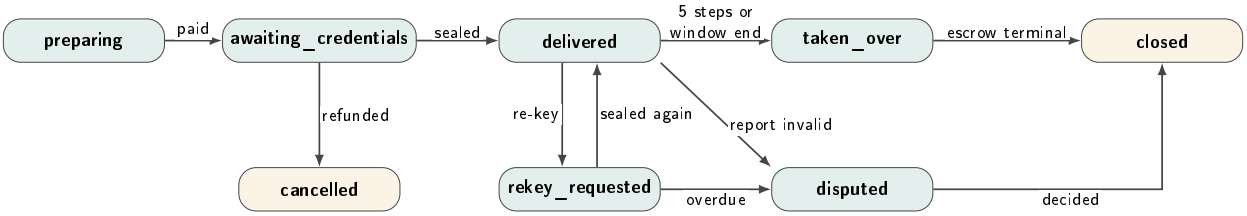


Figure 4: Handover status. Terminal states are shaded. Not drawn: `preparing`, `delivered` and `rekey_requested` also go to `cancelled` on a refund before the delivery event, and a withdrawn dispute returns to `delivered`. The trigger `handover_guard` refuses every other change.

7.3 "The credentials do not work"

Within the takeover window, the buyer may report that the credentials do not work (FR-GAM-14). The service marks the handover `disputed` and opens a `credentials_invalid` dispute in Unit 7 in the same transaction; the window then no longer runs, so the takeover cannot complete while the dispute is open. The report is refused once the buyer has confirmed changing the recovery e-mail, because a buyer who could change it clearly got in. This is the one race in the unit where a person can act against themselves, so it was tested directly: on 20 orders, a report and the recovery-e-mail confirmation were submitted at the same moment. On every order exactly one of them succeeded, never both (three runs; the split between them varied). If the buyer withdraws the dispute, the takeover resumes, and if its window has meanwhile ended, the next job run completes it.

7.4 Protection period and seller recovery

During the protection period the buyer can open a `seller_recovery` dispute with evidence such as the publisher's security e-mails (FR-GAM-16). The buyer can also release the payment early, with step-up authentication, through Unit 2 (FR-GAM-17, SEC-AUTH-08).

When an operator confirms that a seller recovered a sold account, during the protection period or by a report after release, Unit 7 suspends the seller and adds the seller's NID blind index to the blacklist with four-eyes approval, through Unit 1 (FR-GAM-18). For the complaint to law enforcement, `GameAccountQueries.evidence` supplies:

- the listing details, the ownership code and the operator's review, with a signed link to the screenshot;
- the timeline: payment, credential deliveries with key versions, re-key, each confirmed step with its session, and the takeover;
- the credential-access log;
- the fingerprints of the buyer's keys, which show which key each delivery was sealed for.

The evidence contains no ciphertext and no public key. The ciphertext would be useless to anyone but the buyer, and by then it has usually been deleted.

7.5 Deletion

When the order's escrow reaches a terminal state and no dispute is open, the ciphertext is deleted: its bytes are removed and only the deletion time is kept (FR-GAM-19). A refund before the takeover deletes it too. The database refuses to close or cancel a handover while any ciphertext remains, and the only change ever allowed to a sealed-credentials row is this deletion. The public keys, access log and confirmations stay, as evidence (SRS retention schedule).

8 Scheduled Jobs

Each job scans due rows through a partial index and is safe on several instances (`FOR UPDATE SKIP LOCKED`). The deadline refunds themselves are Unit 2's jobs.

Job	Every	Action
Takeover window end	1 min	`complete_if_due`: complete takeovers whose window has ended, at the end time, and record the delivery event (FR-GAM-15).
Re-key overdue	5 min	`rekey_overdue`: move unanswered re-keys to `disputed` and open the dispute (DV-37).
Proof expiry	1 hour	Mark issued codes past DV-33 as `expired`.

The window-end job compares with the statement clock (`clock_timestamp()`) rather than the transaction start, so a job transaction that began a moment before the deadline still sees it pass.

9 Security

9.1 Controls

- **No readable credentials on the server** (SEC-GAM-01). The private key is generated non-extractable in the buyer's browser and never leaves it; the server stores only public keys, `enc` and ciphertexts. No server code path has a decryption key, and no staff endpoint returns a ciphertext.
- **Standard cryptography only** (SEC-DAT-10, C-4): HPKE (RFC 9180) with DHKEM(P-256, HKDF-SHA256), HKDF-SHA256 and AES-256-GCM, through a library built on the Web Cryptography API. The team writes no primitive and no construction.
- **Binding.** Associated data ties each ciphertext to its order and key version; the database ties each ciphertext to an existing key version, and refuses any length that is not padded.
- **Strict pages** (SEC-GAM-03). The credential-entry and credential-view routes load no third-party script, with a nonce-based Content Security Policy and `script-src 'self'` only; no analytics, error-reporting or chat widget is loaded on them. Responses are `no-store`. The fields have autocomplete and spell-checking off.
- **Nothing secret elsewhere** (SEC-DAT-07, FR-NTF-08). Ciphertext, keys and the plaintext never appear in logs, analytics, URLs, notifications or event payloads. Request logging for the sealed endpoints records the size only.
- **Integrity and evidence.** Public keys, the access log and the confirmations are append-only; a sealed-credentials row can only be emptied. The application role cannot update these tables or delete sealed rows (tested).

9.2 Threats considered

Threat	Defence, or limit
Database or backup stolen	Only public keys and ciphertexts: nothing decryptable.
Operator curious or bribed	No endpoint returns ciphertext; it is useless without the buyer's browser anyway; screenshot views audited.
Server swaps the buyer's key	Fingerprints computed in both browsers and compared, ideally by phone. Limit: a fully compromised server could also alter the code it sends (AS-6); reduced by the change controls of SEC-SDL.
Ciphertext moved to another order	Associated data: fails to open (tested).
Seller lists an account he does not control	Fresh code on the live profile; screenshot alone not enough when the profile is public.
Someone blocks another's player ID	Claims are taken only on approval, not on drafting.
Seller recovers the account after sale	Payment held for CP-24 after takeover; `seller_recovery` dispute; suspension, blacklist and evidence (FR-GAM-18).
Buyer falsely says "they do not work"	Refused once the recovery-e-mail step is confirmed; the access log shows every retrieval and the confirmations show what the buyer did.
Publisher bans the account	Not preventable; disclosed and acknowledged before payment (FR-GAM-05).
Malicious browser extension on a user's device	Out of scope: the SRS excludes compromise of a user's own device; the buyer is advised to use a clean browser profile.

10 Parameters

SRS parameters used: CP-22 (credential delivery); CP-23 (takeover window); CP-24 (protection period, applied by Unit 2); CP-31 (signed links); CP-33 (upload size); CP-36 (feature switch).

Design values introduced (continuing DV-1 to DV-32):

ID	Value	Default	Reason
DV-33	Ownership code	`TB-XXXX-XXXX`, 40 random bits (Crockford base32); valid 7 days	Fresh, easy to type into a profile.
DV-34	Sealed payload	length prefix, JSON, zero-padded to 512-byte blocks; at most 4 KB	Hides password length; bounds storage.
DV-35	HPKE suite and context	base mode; DHKEM(P-256, HKDF-SHA256), HKDF-SHA256, AES-256-GCM; info `TrustBazaar/game-credentials/v1`; AAD `order:<id>|key:<v>`	SEC-DAT-10's ECDH option in its standard form.
DV-36	Key fingerprint	first 80 bits of SHA-256 of the raw key; 5 groups of 4 hex digits	Short enough to read out; too long to forge.
DV-37	Re-delivery after a re-key	CP-22 after the request; then a dispute	Same time as the first delivery.
DV-38	Credential view	hidden after 2 min idle; clipboard cleared after 60 s	Shortens exposure on a shared screen.
DV-39	Ownership screenshot retention	2 years after the last order on the listing closes; 30 days after a rejection or an unsold end	Evidence for FR-GAM-18; not in the SRS schedule (I-35).

11 Testing

Level	Cases
Database (done)	70 checks in `05_game_accounts_tests.sql`, each rejection required to fail on its intended rule: game details and platform; ownership codes, proof submission and review, live-profile and expiry rules, claims; disclosure, key format and fixing; the fingerprint vector; delivery before payment, by the wrong party, for the wrong key, unpadded, too long, after the deadline, twice; retrieval and its log; checklist steps, undoing, completion by checklist and by window end; reports before and after the recovery-e-mail step; withdrawal; re-key, re-delivery and overdue; deletion, closing and cancelling; events and grants. The Unit 1 to 4 scripts still pass with this schema loaded.
Concurrency (done)	Six races, three runs each on fresh databases (Appendix B.1): 10 approvals of one player ID (1 approved); 10 deliveries (1 ciphertext); 10 final steps (1 completion); 10 re-keys (1 new key); 20 retrievals (20 log rows); on 20 orders a report against the recovery-e-mail step (never both).
Browser (done)	14 checks of `sealed.js` in real Chromium through Playwright: the private key is non-extractable and survives a reload in IndexedDB; buyer and seller compute the same fingerprint; ciphertext sizes; the password is absent from the ciphertext; tampering, another order, another key version and another key all fail; over-long input is refused; lengths are hidden; each seal uses a fresh `enc`; the fingerprint vector matches the database. The browser's real ciphertext was then accepted by `game_accounts.deliver`.
Planned	Firefox and Safari runs of the browser test (Playwright WebKit and Firefox); the RFC 9180 test vectors for DHKEM(P-256, HKDF-SHA256) through the library; negative authorisation tests for every endpoint (SEC-SDL-09); a CSP test that the credential pages load no third-party origin; a log scan in CI for ciphertext and key material.

12 Notes for Integration

I-29 Unit 2. `GameAccountPort.assertCheckoutReady` is `checkout_ready` plus the CP-36 check. `orders.order.paid` for a game account must carry `fulfilment_due_at` (CP-22), as I-19 already requires. The first delivery calls `recordFulfilmentStep(credentials_delivered)`; completion calls `recordDeliveryEvent(game_takeover, at)` with the recorded completion time, which may be slightly in the past. Escrow-terminal events must say whether the delivery event had happened.

I-30 Unit 3. Publish `listings.listing.cancelled` as well, so a claim can be released. While CP-36 is off, refuse game-account publishing and purchases, and leave game-account listings out of `listings.searchable`. Add password words in Bangla and English to the screening terms.

I-31 Unit 1. `FileVault` gains a purpose for ownership screenshots, on its own volume, with retention DV-39. The CP-36 switch is one of the administrator controls under four-eyes approval (SEC-IRM-04). Suspension and blacklisting for FR-GAM-18 use the existing approvals.

I-32 Unit 6. Notify the seller when credentials are requested and the buyer when they are delivered, and remind both at CP-38 before the credential deadline, the window end and the protection end (FR-NTF-07). No notification contains credentials, keys or fingerprints. For FR-GAM-20 (Could), warn a seller whose chat message looks like login details (for example "password" or "pass" in Bangla or English next to a token, or an e-mail address with another string) and link to the sealed handover. The warning never blocks or stores the message's content.

I-33 Unit 7. The ownership-proof queue (FR-OPS-01) calls `reviewProof`. A `credentials_invalid` dispute is opened in the same transaction as the report, and also when a re-key

is overdue. A withdrawal calls `disputeWithdrawn`. The case file and the FR-GAM-18 export use `evidence`.

I-34 Unit 8. Screens of IF-UI-06 use `sealed.js`; check Web Crypto and IndexedDB before purchase (AS-5); warn the buyer to stay in the same browser; follow the strict-page rules of the security section. The browser module needs `@hpke/core` pinned to an exact version and bundled into the application, not loaded from a CDN, because a CDN would be a third-party script on the credential pages (SEC-GAM-03).

I-35 SRS clarifications proposed:

- SEC-DAT-10: name HPKE (RFC 9180) with the suite of DV-35 as the form of the ECDH option, and note the 318-byte limit of RSA-OAEP-3072;
- FR-GAM-12: a re-key is not possible after the recovery-e-mail step, and the seller must seal again within CP-22, or a dispute is opened;
- FR-GAM-15: at the window end the completion time is the end of the window;
- retention schedule: add ownership screenshots (DV-39);
- add DV-33 to DV-39 to the SRS parameter table.

A Traceability

Requirement	Design element	Section
FR-GAM-01	CP-36 checked at listing and checkout; hidden listings; paid orders continue	5
FR-GAM-02	`details`; no login fields; screening terms	5
FR-GAM-03	Ownership codes; encrypted screenshot; live-profile check; `review_proof`	5.2
FR-GAM-04	`claims`, unique while live, taken on approval, released at escrow end	5.2
FR-GAM-05	Disclosure version and acknowledgement; `checkout_ready`	6
FR-GAM-06	No delivery unit; Unit 2 constraints	4
FR-GAM-07	Non-extractable P-256 key in IndexedDB; public key only to the server	6
FR-GAM-08	Fingerprint computed in both browsers, never by the server	6.3
FR-GAM-09	HPKE seal in the seller's browser; padding	6
FR-GAM-10	Request after payment; delivery refused after CP-22	6
FR-GAM-11	`fetch_sealed`: buyer only; append-only access log	6
FR-GAM-12	One re-key; window restarts; DV-37	6.6
FR-GAM-13	Checklist; confirmations; countdown	7
FR-GAM-14	`report_invalid`; refused after the recovery-e-mail step	7
FR-GAM-15	Completion by five steps or window end; delivery event	7
FR-GAM-16	Protection period held by Unit 2; `seller_recovery` disputes	7
FR-GAM-17	Release at protection end or early with step-up (Unit 2)	7
FR-GAM-18	Evidence; suspension and blacklist through Units 7 and 1	7.4
FR-GAM-19	Ciphertext deleted at the end; trigger refuses closing otherwise	7
FR-GAM-20	Chat warning (Unit 6)	12
FR-DSP-05	`evidence`: proof, access log, confirmations	7.4
FR-NTF-08	No secrets in notifications or events	3.1
SEC-GAM-01	Trust boundary; no decryption key on the server	2.2
SEC-GAM-02	Append-only access log	6
SEC-GAM-03	Strict pages, no third-party scripts, bundled library	12
SEC-DAT-07	Nothing secret in logs, URLs or analytics	12
SEC-DAT-10	HPKE with P-256 and AES-256-GCM	6
IF-UI-06	Screens and their rules	12
IF-SW-07	Web Cryptography API through `@hpke/core`	6
NFR-USE-03	Countdown and protection end on the checklist	7
NFR-MNT-02	Release-rule, race and browser tests	11

Requirement	Design element	Section
DR-07	Details, proofs, keys, ciphertext until deletion, confirmations, access log	4

B Database Schema and Tests

B.1 Test results

Loaded after the Unit 1 to Unit 4 schemas in PostgreSQL 16, using PostgreSQL 15 features only. Every check passed.

```
PASS  FR-GAM-02 the platform must be one the game offers [is not offered for this game]
PASS  player IDs are compared without spaces and case
PASS  DV-33 ownership codes have the fixed format [ownership_proofs_code_check]
PASS  one open ownership proof per listing [proof_one_open]
PASS  a proof cannot be reviewed before the screenshot is submitted [not awaiting review]
PASS  submitting needs the screenshot [proof_submitted_fields]
PASS  the game and player ID are fixed once proof is submitted [fixed once ownership proof]
PASS  an operator cannot approve a proof whose live profile did not match [proof_approve_live]
PASS  FR-GAM-03 a rejection needs a reason [proof_reject_reason]
PASS  FR-GAM-03 approval records the operator and takes the player-ID claim
PASS  ... claim and event written
PASS  a reviewed proof is final [reviewed proof is final]
PASS  FR-GAM-04 one live listing per game and player ID: the second approval is refused [already listed]
PASS  DV-33 an expired code cannot be approved [has expired]
PASS  FR-GAM-07 no key before the disclosure is acknowledged [acknowledge the disclosure first]
PASS  a handover starts as preparing, without a key [starts as preparing]
PASS  FR-GAM-05 with the disclosure acknowledged but no key, checkout is not ready
PASS  FR-GAM-07 a public key must be an uncompressed P-256 point [buyer_keys_public_key_check]
PASS  only the buyer registers a key [only the buyer]
PASS  FR-GAM-07 key version 1 registered; checkout is ready
PASS  ... ready
PASS  the key cannot be swapped once registered [key is fixed at checkout]
PASS  buyer keys are append-only [append-only]
PASS  FR-GAM-08 fingerprint test vector (first 80 bits of SHA-256 of the raw key)
PASS  FR-GAM-10 credentials are not accepted before payment [not expected now]
PASS  only the seller delivers [only the seller]
PASS  ciphertext must be sealed for the buyer's current key [current key is version 1]
PASS  DV-34 an unpadded ciphertext length is refused [sealed_credentials_ciphertext_check]
PASS  DV-34 the payload is at most 4 KB [sealed_credentials_ciphertext_check]
PASS  the encapsulated key must be a P-256 point [sealed_credentials_enc_check]
PASS  FR-GAM-09 the first delivery is the fulfilment step and starts the CP-23 window
PASS  ... window ends 24 hours after delivery
PASS  credentials are delivered once (no second ciphertext without a re-key) [not expected now]
PASS  sealed credentials cannot be altered, only deleted [can only be deleted]
PASS  FR-GAM-10 nothing is accepted after the CP-22 deadline [deadline has passed]
PASS  FR-GAM-11 only the buyer of the order can retrieve the ciphertext [only the buyer]
PASS  FR-GAM-11 every retrieval is logged with user, session and time
PASS  SEC-GAM-02 the access log is append-only [append-only]
PASS  only the buyer confirms takeover steps [only the buyer]
PASS  FR-GAM-13 confirming the password step does not complete the takeover
PASS  FR-GAM-14 after the recovery e-mail step, credentials_invalid is refused [no longer accepted]
PASS  FR-GAM-12 no re-key after the recovery e-mail step [already controls the recovery e-mail]
PASS  a step confirmed twice is recorded once
PASS  a confirmed step cannot be undone [fixed once recorded]
PASS  FR-GAM-15 the fifth step completes the takeover (the delivery event), source checklist
PASS  a completed takeover cannot be cancelled [is not permitted]
PASS  before the window ends, the job does nothing
PASS  FR-GAM-15 at the window end the takeover is treated as complete, at the end time
PASS  FR-GAM-14 a credentials_invalid report inside the window opens the dispute path
PASS  while disputed, no step can be confirmed [not in progress]
PASS  while disputed, the window-end job cannot complete the takeover
PASS  FR-DSP-10 a withdrawn dispute resumes the takeover window
PASS  a checklist completion needs all five confirmations [checklist_complete]
PASS  FR-GAM-12 a re-key issues key version 2 and stops the window
PASS  ... status rekey_requested, no window, re-delivery deadline set
PASS  the seller must seal for the new key, not the old one [current key is version 2]
PASS  FR-GAM-12 re-delivery restarts the window and is not a second fulfilment step
```

```
PASS  the old ciphertext is deleted on re-delivery; only version 2 can be retrieved
PASS  FR-GAM-12 only one re-key per order [one re-key has been used]
PASS  DV-37 a re-key the seller does not answer in time becomes a dispute
PASS  ... status disputed
PASS  FR-GAM-19 at the terminal escrow state the ciphertext is deleted and the access log kept
PASS  FR-GAM-04 after the protection period the player ID can be listed again
PASS  after closing, nothing can be retrieved [no credentials are available]
PASS  a closed handover is final [handover is closed]
PASS  FR-GAM-19 a handover cannot be closed while ciphertext remains [ciphertext must be deleted]
PASS  an order refunded before the takeover cancels the handover
PASS  DC-6 outbox events are named game_accounts.<entity>.<past> [outbox_event_type_check]
PASS  application role cannot rewrite takeover confirmations [permission denied]
PASS  application role cannot delete sealed credentials rows (only blank them) [permission denied]

70 checks passed (05_game_accounts_tests.sql, PostgreSQL 16.15 with the Unit 1-4 schemas loaded).
The Unit 1-4 test scripts still pass with Unit 5 loaded (46, 45, 67 and 93 checks).

Concurrency (05_game_accounts_concurrency.sh). Three runs on fresh databases; the first five
races gave the same counts each time; the split in the last race varies, but 'both' was always 0.
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  1|
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
  t
```

```
  t
  t
  t
  f
  f
  f
  f
  Approval race: 10 operators approve 10 listings of the same player ID at once
    results: approved 1, refused 9
    database: live claims for the player ID=1
  Delivery race: the seller's sealed credentials submitted 10 times at once
    results: accepted 1, refused 9
    database: ciphertexts=1
  Final-step race: the last checklist step confirmed 10 times at once
    results: completed 1, refused or no-op 9
    database: takeover events=1
  Re-key race: 10 re-key requests at once
    results: accepted 1, refused 9
    database: keys=2
  Retrieval storm: 20 retrievals at once
    results: returned 20
    database: access log rows=20
  Report-or-confirm race: 20 orders, each with an invalid-report and the e-mail step at once
    results: reports accepted 12, e-mail steps accepted 8
    database: disputed=12, e-mail confirmed=8, both=0

Browser (crypto/browser_test.mjs: sealed.js in real Chromium):
  PASS  FR-GAM-07 the private key is non-extractable
  PASS  FR-GAM-07 the public key is a 65-byte uncompressed P-256 point
  PASS  FR-GAM-08 buyer and seller compute the same fingerprint independently (eac3-2d92-149c-c061-d7e2)
  PASS  DV-34 ciphertext is 512 k + 16 bytes; enc is a 65-byte point (528 bytes)
  PASS  SEC-GAM-01 the ciphertext does not contain the password in clear
  PASS  FR-GAM-11 after a reload the stored key (still non-extractable) opens the credentials
  PASS  a modified ciphertext is rejected (AES-GCM tag)
  PASS  ciphertext moved to another order is rejected (associated data)
  PASS  ciphertext replayed under another key version is rejected
  PASS  another browser's key cannot open it
  PASS  DV-34 credentials over 4 KB are refused before sealing
  PASS  DV-34 a 5-character and a 300-character password give the same ciphertext size (528 bytes)
  PASS  each sealing uses a fresh encapsulated key
  PASS  FR-GAM-08 browser fingerprint of the test vector equals the SQL one (0ed3-a6ab-957f-f6f5-9a96)
  14 of 14 browser checks passed (Chromium via Playwright 1.56.0, @hpke/core 1.9.0)
  The browser's real ciphertext for the same key was then accepted by game_accounts.deliver.
```

B.2 Schema

```
-- ================================================================
-- TrustBazaar - Unit 5: Game-Account Escrow - PostgreSQL 15 schema
-- Design document: docs/design/05_game_accounts.pdf
-- Requirements: SRS v2.0 FR-GAM-01..20; FR-PAY-07/08 (game side); FR-DSP-01 and FR-DSP-05
--   (game windows and evidence); SEC-GAM-01..03, SEC-DAT-07, SEC-DAT-10; IF-UI-06, IF-SW-07; DR-07.
-- Depends on: 01_identity_access.sql (identity.users, identity.staff_accounts, identity.sessions,
--   identity.forbid_change), 02_orders_payments.sql (orders.orders), 03_listings_auctions.sql
--   (listings.listings). Unit 4 is not used: game accounts have no delivery (FR-GAM-06).
-- Conventions DC-1..DC-14. No function here reads another unit's tables (DC-1); order and
-- listing facts arrive as arguments. The server never holds anything that can decrypt the
-- credentials (SEC-GAM-01): it stores the buyer's public key and the sealed ciphertext only.
-- ================================================================

CREATE SCHEMA IF NOT EXISTS game_accounts;

-- ---------------------------------------------------------------- types
CREATE TYPE game_accounts.proof_status    AS ENUM ('issued', 'submitted', 'approved', 'rejected', 'expired');
CREATE TYPE game_accounts.live_check      AS ENUM ('matched', 'not_matched', 'not_available');
CREATE TYPE game_accounts.handover_status AS ENUM
  ('preparing',             -- before payment: disclosure acknowledged, buyer key registered
   'awaiting_credentials',  -- paid; the seller must seal the credentials by CP-22
   'delivered',             -- sealed credentials delivered; takeover window CP-23 running
   'rekey_requested',       -- the buyer lost the key; waiting for the seller to seal again (FR-GAM-12)
   'disputed',              -- credentials_invalid reported; decided in Unit 7
   'taken_over',            -- takeover complete: the delivery event (FR-GAM-15); protection period runs
   'cancelled',             -- refunded before the delivery event
   'closed');               -- escrow terminal after delivery; ciphertext deleted (FR-GAM-19)
```

```
CREATE TYPE game_accounts.takeover_step   AS ENUM
  ('password', 'recovery_email', 'recovery_phone', 'social_logins', 'two_step');   -- FR-GAM-13
CREATE TYPE game_accounts.takeover_source AS ENUM ('checklist', 'window_end');

-- ====================================================================
-- Games and listing details (FR-GAM-02). A listing's title, description and price are Unit 3's;
-- this unit adds the structured game fields. No login detail is ever part of a listing.
-- ====================================================================
CREATE TABLE game_accounts.games (
  id                smallint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  slug              varchar(40) NOT NULL UNIQUE CHECK (slug ~ '^[a-z0-9-]+$'),
  name              varchar(80) NOT NULL,
  platforms         text[] NOT NULL CHECK (cardinality(platforms) >= 1),
  code_placement    varchar(200) NOT NULL,      -- where the ownership code goes, e.g. "profile bio"
  publisher_note    varchar(500) NOT NULL,      -- the publisher's stance on transfers, for the disclosure
  active            boolean NOT NULL DEFAULT true
);

CREATE TABLE game_accounts.details (
  listing_id      uuid PRIMARY KEY REFERENCES listings.listings(id),
  seller_id       uuid NOT NULL REFERENCES identity.users(id),
  game_id         smallint NOT NULL REFERENCES game_accounts.games(id),
  platform        varchar(30) NOT NULL,
  player_id       varchar(64) NOT NULL CHECK (player_id ~ '^[[:print:]]+$' AND length(btrim(player_id)) >= 2)
    ,
  player_id_norm  varchar(64) GENERATED ALWAYS AS (lower(regexp_replace(player_id, '\s', '', 'g'))) STORED,
  contents        varchar(2000) NOT NULL CHECK (length(btrim(contents)) >= 10),   -- level, items, linked
    purchases
  created_at      timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX details_player ON game_accounts.details (game_id, player_id_norm);

CREATE FUNCTION game_accounts.details_guard() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM game_accounts.games g WHERE g.id = NEW.game_id AND NEW.platform::text = ANY (g
    .platforms)) THEN
    RAISE EXCEPTION 'platform␣%␣is␣not␣offered␣for␣this␣game', NEW.platform USING ERRCODE = 'check_violation'
    ;
  END IF;
  IF TG_OP = 'UPDATE' AND (NEW.game_id, NEW.player_id, NEW.seller_id) IS DISTINCT FROM (OLD.game_id, OLD.
    player_id, OLD.seller_id)
     AND EXISTS (SELECT 1 FROM game_accounts.ownership_proofs WHERE listing_id = OLD.listing_id AND status IN
      ('submitted', 'approved')) THEN
    RAISE EXCEPTION 'game␣and␣player␣ID␣are␣fixed␣once␣ownership␣proof␣is␣submitted' USING ERRCODE = '
    check_violation';
  END IF;
  RETURN NEW;
END $$;

-- ====================================================================
-- Ownership proof (FR-GAM-03). The code is not secret: it is placed publicly in the game profile.
-- What it proves is freshness and control. The screenshot is Confidential and stored encrypted
-- like KYC files (Unit 1 FileVault envelope: per-file key wrapped by the KEK).
-- ====================================================================
CREATE TABLE game_accounts.ownership_proofs (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_id        uuid NOT NULL REFERENCES game_accounts.details(listing_id),
  code              varchar(12) NOT NULL UNIQUE CHECK (code ~ '^TB-[0-9A-HJKMNP-TV-Z]{4}-[0-9A-HJKMNP-TV-Z
    ]{4}$'),
  status            game_accounts.proof_status NOT NULL DEFAULT 'issued',
  issued_at         timestamptz NOT NULL DEFAULT now(),
  expires_at        timestamptz NOT NULL,
  screenshot_key    varchar(80) UNIQUE,               -- random name on the evidence volume
  screenshot_dek    bytea,                            -- wrapped data key; NULL once erased
  screenshot_sha256 bytea CHECK (octet_length(screenshot_sha256) = 32),
  submitted_at      timestamptz,
  reviewed_by       uuid REFERENCES identity.staff_accounts(id),
  reviewed_at       timestamptz,
  live_check        game_accounts.live_check,
  review_reason     text,
  CONSTRAINT proof_expiry CHECK (expires_at > issued_at),
  CONSTRAINT proof_submitted_fields CHECK ((status IN ('submitted', 'approved', 'rejected'))
        = (submitted_at IS NOT NULL AND screenshot_key IS NOT NULL AND screenshot_sha256 IS NOT NULL) OR
    status = 'expired'),
  CONSTRAINT proof_review_fields CHECK ((status IN ('approved', 'rejected'))
```

```
        = (reviewed_by IS NOT NULL AND reviewed_at IS NOT NULL AND live_check IS NOT NULL)),
  CONSTRAINT proof_reject_reason CHECK (status <> 'rejected' OR length(btrim(coalesce(review_reason, ''))) >=
      10),
  CONSTRAINT proof_approve_live CHECK (status <> 'approved' OR live_check <> 'not_matched')
);
CREATE UNIQUE INDEX proof_one_open ON game_accounts.ownership_proofs (listing_id) WHERE status IN ('issued',
    'submitted');
CREATE INDEX proofs_queue ON game_accounts.ownership_proofs (submitted_at) WHERE status = 'submitted';

CREATE TRIGGER details_guard BEFORE INSERT OR UPDATE ON game_accounts.details
  FOR EACH ROW EXECUTE FUNCTION game_accounts.details_guard();

CREATE FUNCTION game_accounts.proof_guard() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    IF NEW.status <> 'issued' THEN RAISE EXCEPTION 'a␣proof␣starts␣as␣issued' USING ERRCODE = '
    check_violation'; END IF;
    RETURN NEW;
  END IF;
  IF (NEW.listing_id, NEW.code, NEW.issued_at, NEW.expires_at) IS DISTINCT FROM (OLD.listing_id, OLD.code,
    OLD.issued_at, OLD.expires_at) THEN
    RAISE EXCEPTION 'proof␣code␣and␣validity␣are␣fixed' USING ERRCODE = 'check_violation';
  END IF;
  IF NEW.status IS DISTINCT FROM OLD.status AND NOT ((OLD.status, NEW.status) IN (
      ('issued'::game_accounts.proof_status, 'submitted'::game_accounts.proof_status), ('issued', 'expired')
    ,
      ('submitted', 'approved'), ('submitted', 'rejected'))) THEN
    RAISE EXCEPTION 'proof␣%␣->␣%␣is␣not␣permitted', OLD.status, NEW.status USING ERRCODE = 'check_violation'
    ;
  END IF;
  IF OLD.status IN ('approved', 'rejected') AND (NEW.reviewed_by, NEW.reviewed_at, NEW.live_check, NEW.
    review_reason, NEW.screenshot_key, NEW.screenshot_sha256)
      IS DISTINCT FROM (OLD.reviewed_by, OLD.reviewed_at, OLD.live_check, OLD.review_reason, OLD.
    screenshot_key, OLD.screenshot_sha256) THEN
    RAISE EXCEPTION 'a␣reviewed␣proof␣is␣final' USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER proof_guard BEFORE INSERT OR UPDATE ON game_accounts.ownership_proofs
  FOR EACH ROW EXECUTE FUNCTION game_accounts.proof_guard();

-- ======================================================================
-- Player-ID claims (FR-GAM-04). Taken when ownership is approved, not when a draft is written,
-- so nobody can block a player ID by drafting it. Held while the listing is live and, after a
-- sale, until the order's escrow is terminal (takeover window and protection period included).
-- ======================================================================
CREATE TABLE game_accounts.claims (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  game_id          smallint NOT NULL REFERENCES game_accounts.games(id),
  player_id_norm   varchar(64) NOT NULL,
  listing_id       uuid NOT NULL REFERENCES game_accounts.details(listing_id),
  claimed_at       timestamptz NOT NULL DEFAULT now(),
  order_id         uuid REFERENCES orders.orders(id),          -- set when sold
  released_at      timestamptz,
  release_reason   varchar(40) CHECK (release_reason IN ('listing_ended', 'escrow_terminal', 'operator')),
  CONSTRAINT claim_release_fields CHECK ((released_at IS NULL) = (release_reason IS NULL))
);
CREATE UNIQUE INDEX claims_one_live ON game_accounts.claims (game_id, player_id_norm) WHERE released_at IS
    NULL;

-- ======================================================================
-- Handover per order (FR-GAM-05 to FR-GAM-17)
-- ======================================================================
CREATE TABLE game_accounts.handovers (
  order_id             uuid PRIMARY KEY REFERENCES orders.orders(id),
  listing_id           uuid NOT NULL REFERENCES game_accounts.details(listing_id),
  buyer_id             uuid NOT NULL REFERENCES identity.users(id),
  seller_id            uuid NOT NULL REFERENCES identity.users(id),
  status               game_accounts.handover_status NOT NULL DEFAULT 'preparing',
  disclosure_version   smallint NOT NULL CHECK (disclosure_version >= 1),       -- FR-GAM-05
  disclosure_ack_at    timestamptz NOT NULL DEFAULT now(),
  key_version          smallint NOT NULL DEFAULT 0 CHECK (key_version BETWEEN 0 AND 2),
  rekeys_used          smallint NOT NULL DEFAULT 0 CHECK (rekeys_used BETWEEN 0 AND 1),   -- FR-GAM-12: one
    re-key
  credentials_due_at   timestamptz,                    -- CP-22 (the order's fulfilment deadline)
```

```
  first_delivered_at   timestamptz,
  delivered_at         timestamptz,                    -- latest delivery (restarts the window, FR-GAM-12)
  takeover_due_at      timestamptz,                    -- delivered_at + CP-23
  rekey_requested_at   timestamptz,
  rekey_due_at         timestamptz,                    -- DV-37
  pw_changed_at        timestamptz,
  recovery_email_at    timestamptz,
  recovery_phone_at    timestamptz,
  social_logins_at     timestamptz,
  two_step_at          timestamptz,
  taken_over_at        timestamptz,
  takeover_source      game_accounts.takeover_source,
  disputed_at          timestamptz,
  cancelled_at         timestamptz,
  closed_at            timestamptz,
  created_at           timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT handover_parties CHECK (buyer_id <> seller_id),
  CONSTRAINT key_version_counts_rekeys CHECK (key_version = 0 OR key_version = rekeys_used + 1),
  CONSTRAINT paid_has_deadline CHECK (status IN ('preparing', 'cancelled') OR credentials_due_at IS NOT NULL)
    ,
  CONSTRAINT paid_needs_key CHECK (status IN ('preparing', 'cancelled') OR key_version >= 1),
  CONSTRAINT delivered_fields CHECK ((status = 'delivered') <= (delivered_at IS NOT NULL AND takeover_due_at
    IS NOT NULL)),
  CONSTRAINT rekey_fields CHECK ((status = 'rekey_requested') = (rekey_due_at IS NOT NULL)),
  CONSTRAINT takeover_fields CHECK ((taken_over_at IS NULL) = (takeover_source IS NULL)
        AND (status <> 'taken_over' OR taken_over_at IS NOT NULL)),
  CONSTRAINT checklist_complete CHECK (takeover_source IS DISTINCT FROM 'checklist' OR
        (pw_changed_at IS NOT NULL AND recovery_email_at IS NOT NULL AND recovery_phone_at IS NOT NULL
         AND social_logins_at IS NOT NULL AND two_step_at IS NOT NULL)),
  CONSTRAINT cancelled_fields CHECK ((status = 'cancelled') = (cancelled_at IS NOT NULL)),
  CONSTRAINT closed_fields CHECK ((status = 'closed') = (closed_at IS NOT NULL)),
  CONSTRAINT cancel_only_before_takeover CHECK (status <> 'cancelled' OR taken_over_at IS NULL)
);
CREATE INDEX handovers_takeover_due ON game_accounts.handovers (takeover_due_at) WHERE status = 'delivered';
CREATE INDEX handovers_rekey_due ON game_accounts.handovers (rekey_due_at) WHERE status = 'rekey_requested';

CREATE FUNCTION game_accounts.handover_guard() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    IF NEW.status <> 'preparing' OR NEW.key_version <> 0 THEN
      RAISE EXCEPTION 'a␣handover␣starts␣as␣preparing,␣without␣a␣key' USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
  END IF;
  IF (NEW.order_id, NEW.listing_id, NEW.buyer_id, NEW.seller_id, NEW.disclosure_version, NEW.
     disclosure_ack_at)
     IS DISTINCT FROM (OLD.order_id, OLD.listing_id, OLD.buyer_id, OLD.seller_id, OLD.disclosure_version, OLD
     .disclosure_ack_at) THEN
    RAISE EXCEPTION 'handover␣parties␣and␣disclosure␣are␣fixed' USING ERRCODE = 'check_violation';
  END IF;
  IF OLD.status IN ('cancelled', 'closed') THEN
    RAISE EXCEPTION 'handover␣is␣closed' USING ERRCODE = 'check_violation';
  END IF;
  IF NEW.status IS DISTINCT FROM OLD.status AND NOT ((OLD.status, NEW.status) IN (
       ('preparing'::game_accounts.handover_status, 'awaiting_credentials'::game_accounts.handover_status), (
     'preparing', 'cancelled'),
       ('awaiting_credentials', 'delivered'), ('awaiting_credentials', 'cancelled'),
       ('delivered', 'taken_over'), ('delivered', 'disputed'), ('delivered', 'rekey_requested'), ('delivered'
     , 'cancelled'),
       ('rekey_requested', 'delivered'), ('rekey_requested', 'disputed'), ('rekey_requested', 'cancelled'),
       ('disputed', 'delivered'), ('disputed', 'closed'),
       ('taken_over', 'closed'))) THEN
    RAISE EXCEPTION 'handover␣%␣->␣%␣is␣not␣permitted', OLD.status, NEW.status USING ERRCODE = '
     check_violation';
  END IF;
  -- a confirmed checklist step is never undone; the takeover moment is never moved
  IF (OLD.pw_changed_at IS NOT NULL AND NEW.pw_changed_at IS DISTINCT FROM OLD.pw_changed_at)
     OR (OLD.recovery_email_at IS NOT NULL AND NEW.recovery_email_at IS DISTINCT FROM OLD.recovery_email_at)
     OR (OLD.recovery_phone_at IS NOT NULL AND NEW.recovery_phone_at IS DISTINCT FROM OLD.recovery_phone_at)
     OR (OLD.social_logins_at IS NOT NULL AND NEW.social_logins_at IS DISTINCT FROM OLD.social_logins_at)
     OR (OLD.two_step_at IS NOT NULL AND NEW.two_step_at IS DISTINCT FROM OLD.two_step_at)
     OR (OLD.taken_over_at IS NOT NULL AND NEW.taken_over_at IS DISTINCT FROM OLD.taken_over_at)
     OR (OLD.first_delivered_at IS NOT NULL AND NEW.first_delivered_at IS DISTINCT FROM OLD.
     first_delivered_at) THEN
```

```
    RAISE EXCEPTION 'takeover␣confirmations␣and␣the␣takeover␣time␣are␣fixed␣once␣recorded' USING ERRCODE = '
     check_violation';
  END IF;
  IF NEW.rekeys_used < OLD.rekeys_used OR NEW.key_version < OLD.key_version THEN
    RAISE EXCEPTION 'key␣versions␣only␣increase' USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER handover_guard BEFORE INSERT OR UPDATE ON game_accounts.handovers
  FOR EACH ROW EXECUTE FUNCTION game_accounts.handover_guard();

-- the buyer's public keys (FR-GAM-07): raw uncompressed P-256 points (65 bytes, 0x04 prefix).
-- The private key never leaves the buyer's browser, where it is non-extractable.
CREATE TABLE game_accounts.buyer_keys (
  order_id     uuid NOT NULL REFERENCES game_accounts.handovers(order_id),
  version      smallint NOT NULL CHECK (version BETWEEN 1 AND 2),
  public_key   bytea NOT NULL CHECK (octet_length(public_key) = 65 AND get_byte(public_key, 0) = 4),
  session_id   uuid NOT NULL REFERENCES identity.sessions(id),       -- the browser session that registered
     it
  created_at   timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (order_id, version)
);
CREATE TRIGGER buyer_keys_append_only BEFORE UPDATE OR DELETE ON game_accounts.buyer_keys
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();

-- the sealed credentials (FR-GAM-09): HPKE base mode, DHKEM(P-256, HKDF-SHA256), HKDF-SHA256,
-- AES-256-GCM (DV-35). enc = the sender's encapsulated key (65 bytes). The plaintext is padded
-- to a multiple of 512 bytes before sealing (DV-34), so ciphertext = 512 k + 16-byte tag.
CREATE TABLE game_accounts.sealed_credentials (
  order_id      uuid NOT NULL,
  key_version   smallint NOT NULL,
  enc           bytea CHECK (octet_length(enc) = 65 AND get_byte(enc, 0) = 4),
  ciphertext    bytea CHECK (octet_length(ciphertext) BETWEEN 528 AND 4112 AND (octet_length(ciphertext) -
     16) % 512 = 0),
  delivered_at  timestamptz NOT NULL DEFAULT now(),
  deleted_at    timestamptz,
  PRIMARY KEY (order_id, key_version),
  FOREIGN KEY (order_id, key_version) REFERENCES game_accounts.buyer_keys(order_id, version),
  CONSTRAINT sealed_deleted CHECK ((deleted_at IS NULL) = (enc IS NOT NULL AND ciphertext IS NOT NULL))
);

CREATE FUNCTION game_accounts.sealed_guard() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.deleted_at IS NULL OR OLD.deleted_at IS NOT NULL
     OR (NEW.order_id, NEW.key_version, NEW.delivered_at) IS DISTINCT FROM (OLD.order_id, OLD.key_version,
     OLD.delivered_at) THEN
    RAISE EXCEPTION 'sealed␣credentials␣can␣only␣be␣deleted' USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER sealed_guard BEFORE UPDATE ON game_accounts.sealed_credentials
  FOR EACH ROW EXECUTE FUNCTION game_accounts.sealed_guard();

-- credential-access log (FR-GAM-11, SEC-GAM-02): append-only
CREATE TABLE game_accounts.access_log (
  id           bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id     uuid NOT NULL REFERENCES game_accounts.handovers(order_id),
  user_id      uuid NOT NULL REFERENCES identity.users(id),
  session_id   uuid NOT NULL REFERENCES identity.sessions(id),
  key_version  smallint NOT NULL,
  source_ip    inet,
  at           timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX access_log_order ON game_accounts.access_log (order_id, id);
CREATE TRIGGER access_log_append_only BEFORE UPDATE OR DELETE ON game_accounts.access_log
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();

-- takeover confirmations (DR-07): append-only, one per step
CREATE TABLE game_accounts.confirmations (
  order_id     uuid NOT NULL REFERENCES game_accounts.handovers(order_id),
  step         game_accounts.takeover_step NOT NULL,
  session_id   uuid NOT NULL REFERENCES identity.sessions(id),
  at           timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (order_id, step)
);
```

```
CREATE TRIGGER confirmations_append_only BEFORE UPDATE OR DELETE ON game_accounts.confirmations
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();

CREATE TABLE game_accounts.outbox (                                          -- DC-6
  id              bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  event_type      varchar(80) NOT NULL CHECK (event_type ~ '^game_accounts\.[a-z_]+\.[a-z_]+$'),
  aggregate_id    uuid NOT NULL,
  payload         jsonb NOT NULL,
  created_at      timestamptz NOT NULL DEFAULT now(),
  attempts        smallint NOT NULL DEFAULT 0,
  next_attempt_at timestamptz NOT NULL DEFAULT now(),
  dispatched_at   timestamptz
);
CREATE INDEX game_outbox_pending ON game_accounts.outbox (next_attempt_at) WHERE dispatched_at IS NULL;

-- ======================================================================
-- Reference functions
-- ======================================================================
-- FR-GAM-08: fingerprint of a public key: the first 80 bits of SHA-256 over the raw key, as five
-- groups of four hex digits (DV-36). Browsers compute it themselves; this is the reference and
-- the test vector. A fingerprint shown by the server proves nothing, so the API never sends one.
CREATE FUNCTION game_accounts.fingerprint(p_key bytea) RETURNS text LANGUAGE sql IMMUTABLE AS $$
  SELECT array_to_string(ARRAY(SELECT substr(h, i, 4) FROM generate_series(1, 17, 4) i), '-')
    FROM (SELECT encode(sha256(p_key), 'hex') AS h) x
$$;

-- ======================================================================
-- Operations. Each locks the handover row; the service has already locked the order through
-- OrdersPort.lockForFulfilment (DC-14), and calls EscrowCommands afterwards.
-- ======================================================================

-- FR-GAM-03: review an ownership proof; approval takes the player-ID claim (FR-GAM-04)
CREATE FUNCTION game_accounts.review_proof(p_proof uuid, p_staff uuid, p_approve boolean,
                                           p_live game_accounts.live_check, p_reason text)
RETURNS game_accounts.proof_status LANGUAGE plpgsql AS $$
DECLARE p game_accounts.ownership_proofs; d game_accounts.details;
BEGIN
  SELECT * INTO p FROM game_accounts.ownership_proofs WHERE id = p_proof FOR UPDATE;
  IF NOT FOUND OR p.status <> 'submitted' THEN
    RAISE EXCEPTION 'proof␣is␣not␣awaiting␣review' USING ERRCODE = 'check_violation';
  END IF;
  IF p_approve AND now() >= p.expires_at THEN
    RAISE EXCEPTION 'ownership␣code␣has␣expired;␣the␣seller␣must␣request␣a␣new␣one' USING ERRCODE = '
     check_violation';
  END IF;
  SELECT * INTO d FROM game_accounts.details WHERE listing_id = p.listing_id;
  UPDATE game_accounts.ownership_proofs
     SET status = CASE WHEN p_approve THEN 'approved'::game_accounts.proof_status ELSE 'rejected' END,
         reviewed_by = p_staff, reviewed_at = now(), live_check = p_live, review_reason = p_reason
   WHERE id = p_proof;
  IF p_approve THEN
    BEGIN
      INSERT INTO game_accounts.claims (game_id, player_id_norm, listing_id)
      VALUES (d.game_id, d.player_id_norm, d.listing_id);
    EXCEPTION WHEN unique_violation THEN
      RAISE EXCEPTION 'this␣player␣ID␣is␣already␣listed,␣or␣its␣last␣sale␣is␣still␣protected' USING ERRCODE =
      'check_violation';
    END;
  END IF;
  INSERT INTO game_accounts.outbox (event_type, aggregate_id, payload)
  VALUES (CASE WHEN p_approve THEN 'game_accounts.ownership.approved' ELSE 'game_accounts.ownership.rejected'
     END,
          p.listing_id, jsonb_build_object('proofId', p_proof, 'reason', p_reason));
  RETURN CASE WHEN p_approve THEN 'approved'::game_accounts.proof_status ELSE 'rejected' END;
END $$;

-- FR-GAM-05, FR-GAM-07: register (or replace, before payment) the buyer's public key
CREATE FUNCTION game_accounts.register_key(p_order uuid, p_buyer uuid, p_key bytea, p_session uuid)
RETURNS smallint LANGUAGE plpgsql AS $$
DECLARE h game_accounts.handovers;
BEGIN
  SELECT * INTO h FROM game_accounts.handovers WHERE order_id = p_order FOR UPDATE;
  IF NOT FOUND THEN RAISE EXCEPTION 'acknowledge␣the␣disclosure␣first' USING ERRCODE = 'check_violation'; END
     IF;
```

```
  IF p_buyer <> h.buyer_id THEN RAISE EXCEPTION 'only␣the␣buyer␣registers␣a␣key' USING ERRCODE = '
     insufficient_privilege'; END IF;
  IF h.status <> 'preparing' OR h.key_version <> 0 THEN
    RAISE EXCEPTION 'the␣key␣is␣fixed␣at␣checkout;␣after␣payment␣use␣a␣re-key' USING ERRCODE = '
     check_violation';
  END IF;
  INSERT INTO game_accounts.buyer_keys (order_id, version, public_key, session_id) VALUES (p_order, 1, p_key,
      p_session);
  UPDATE game_accounts.handovers SET key_version = 1 WHERE order_id = p_order;
  RETURN 1;
END $$;

-- Unit 2's GameAccountPort.assertCheckoutReady (FR-GAM-05, FR-GAM-07)
CREATE FUNCTION game_accounts.checkout_ready(p_order uuid) RETURNS boolean LANGUAGE sql STABLE AS $$
  SELECT EXISTS (SELECT 1 FROM game_accounts.handovers WHERE order_id = p_order AND status = 'preparing' AND
     key_version = 1)
$$;

-- order paid: the seller is asked for the credentials (FR-GAM-10)
CREATE FUNCTION game_accounts.mark_paid(p_order uuid, p_due timestamptz) RETURNS void LANGUAGE plpgsql AS $$
BEGIN
  UPDATE game_accounts.handovers SET status = 'awaiting_credentials', credentials_due_at = p_due
   WHERE order_id = p_order AND status = 'preparing';
  IF NOT FOUND THEN RAISE EXCEPTION 'handover␣is␣not␣ready␣for␣payment' USING ERRCODE = 'check_violation';
     END IF;
  INSERT INTO game_accounts.outbox (event_type, aggregate_id, payload)
  VALUES ('game_accounts.credentials.requested', p_order, jsonb_build_object('dueAt', p_due));
END $$;

-- FR-GAM-09, FR-GAM-10, FR-GAM-12, FR-GAM-13: the seller delivers sealed credentials for the
-- buyer's current key. Returns true on the first delivery (the fulfilment step for Unit 2).
CREATE FUNCTION game_accounts.deliver(p_order uuid, p_seller uuid, p_key_version smallint,
                                      p_enc bytea, p_ciphertext bytea, p_window interval)
RETURNS boolean LANGUAGE plpgsql AS $$
DECLARE h game_accounts.handovers;
BEGIN
  IF p_window <= interval '0' THEN RAISE EXCEPTION 'CP-23␣must␣be␣positive' USING ERRCODE = 'check_violation'
     ; END IF;
  SELECT * INTO h FROM game_accounts.handovers WHERE order_id = p_order FOR UPDATE;
  IF NOT FOUND THEN RAISE EXCEPTION 'no␣handover␣for␣this␣order' USING ERRCODE = 'check_violation'; END IF;
  IF p_seller <> h.seller_id THEN RAISE EXCEPTION 'only␣the␣seller␣delivers␣credentials' USING ERRCODE = '
     insufficient_privilege'; END IF;
  IF h.status NOT IN ('awaiting_credentials', 'rekey_requested') THEN
    RAISE EXCEPTION 'credentials␣are␣not␣expected␣now␣(%)', h.status USING ERRCODE = 'check_violation';
  END IF;
  IF h.status = 'awaiting_credentials' AND now() >= h.credentials_due_at THEN
    RAISE EXCEPTION 'the␣credential␣delivery␣deadline␣has␣passed' USING ERRCODE = 'check_violation';
  END IF;
  IF h.status = 'rekey_requested' AND now() >= h.rekey_due_at THEN
    RAISE EXCEPTION 'the␣re-delivery␣deadline␣has␣passed' USING ERRCODE = 'check_violation';
  END IF;
  IF p_key_version <> h.key_version THEN
    RAISE EXCEPTION 'sealed␣for␣key␣version␣%,␣but␣the␣buyer''s␣current␣key␣is␣version␣%', p_key_version, h.
     key_version
      USING ERRCODE = 'check_violation';
  END IF;
  -- older ciphertexts are useless to the buyer and are deleted at once
  UPDATE game_accounts.sealed_credentials SET enc = NULL, ciphertext = NULL, deleted_at = now()
   WHERE order_id = p_order AND deleted_at IS NULL;
  INSERT INTO game_accounts.sealed_credentials (order_id, key_version, enc, ciphertext)
  VALUES (p_order, p_key_version, p_enc, p_ciphertext);
  UPDATE game_accounts.handovers
     SET status = 'delivered', first_delivered_at = coalesce(first_delivered_at, now()),
         delivered_at = now(), takeover_due_at = now() + p_window, rekey_due_at = NULL
   WHERE order_id = p_order;
  INSERT INTO game_accounts.outbox (event_type, aggregate_id, payload)
  VALUES ('game_accounts.credentials.delivered', p_order,
          jsonb_build_object('keyVersion', p_key_version, 'takeoverDueAt', now() + p_window));
  RETURN h.first_delivered_at IS NULL;
END $$;

-- FR-GAM-11: the buyer retrieves the ciphertext; every retrieval is logged
CREATE FUNCTION game_accounts.fetch_sealed(p_order uuid, p_user uuid, p_session uuid, p_ip inet)
RETURNS TABLE (key_version smallint, enc bytea, ciphertext bytea) LANGUAGE plpgsql AS $$
DECLARE h game_accounts.handovers;
```

```
BEGIN
  SELECT * INTO h FROM game_accounts.handovers WHERE order_id = p_order FOR SHARE;
  IF NOT FOUND OR p_user <> h.buyer_id THEN
    RAISE EXCEPTION 'only␣the␣buyer␣of␣this␣order␣can␣retrieve␣the␣credentials' USING ERRCODE = '
     insufficient_privilege';
  END IF;
  IF NOT EXISTS (SELECT 1 FROM game_accounts.sealed_credentials s WHERE s.order_id = p_order AND s.deleted_at
      IS NULL) THEN
    RAISE EXCEPTION 'no␣credentials␣are␣available' USING ERRCODE = 'no_data_found';
  END IF;
  INSERT INTO game_accounts.access_log (order_id, user_id, session_id, key_version, source_ip)
  SELECT p_order, p_user, p_session, s.key_version, p_ip FROM game_accounts.sealed_credentials s
   WHERE s.order_id = p_order AND s.deleted_at IS NULL;
  RETURN QUERY SELECT s.key_version, s.enc, s.ciphertext FROM game_accounts.sealed_credentials s
               WHERE s.order_id = p_order AND s.deleted_at IS NULL;
END $$;

-- FR-GAM-13, FR-GAM-15: the buyer confirms a checklist step. Returns true when this completes the
-- takeover (all five steps): the delivery event for Unit 2.
CREATE FUNCTION game_accounts.confirm_step(p_order uuid, p_buyer uuid, p_step game_accounts.takeover_step,
    p_session uuid)
RETURNS boolean LANGUAGE plpgsql AS $$
DECLARE h game_accounts.handovers;
BEGIN
  SELECT * INTO h FROM game_accounts.handovers WHERE order_id = p_order FOR UPDATE;
  IF NOT FOUND OR p_buyer <> h.buyer_id THEN
    RAISE EXCEPTION 'only␣the␣buyer␣confirms␣the␣takeover' USING ERRCODE = 'insufficient_privilege';
  END IF;
  IF h.status <> 'delivered' THEN RAISE EXCEPTION 'takeover␣is␣not␣in␣progress␣(%)', h.status USING ERRCODE =
      'check_violation'; END IF;
  IF now() >= h.takeover_due_at THEN RAISE EXCEPTION 'the␣takeover␣window␣has␣ended' USING ERRCODE = '
     check_violation'; END IF;
  INSERT INTO game_accounts.confirmations (order_id, step, session_id) VALUES (p_order, p_step, p_session)
  ON CONFLICT DO NOTHING;
  UPDATE game_accounts.handovers SET
      pw_changed_at     = CASE WHEN p_step = 'password'       THEN coalesce(pw_changed_at, now())     ELSE
     pw_changed_at END,
      recovery_email_at = CASE WHEN p_step = 'recovery_email' THEN coalesce(recovery_email_at, now()) ELSE
     recovery_email_at END,
      recovery_phone_at = CASE WHEN p_step = 'recovery_phone' THEN coalesce(recovery_phone_at, now()) ELSE
     recovery_phone_at END,
      social_logins_at  = CASE WHEN p_step = 'social_logins'  THEN coalesce(social_logins_at, now())  ELSE
     social_logins_at END,
      two_step_at       = CASE WHEN p_step = 'two_step'       THEN coalesce(two_step_at, now())       ELSE
     two_step_at END
   WHERE order_id = p_order RETURNING * INTO h;
  IF h.pw_changed_at IS NOT NULL AND h.recovery_email_at IS NOT NULL AND h.recovery_phone_at IS NOT NULL
     AND h.social_logins_at IS NOT NULL AND h.two_step_at IS NOT NULL THEN
    UPDATE game_accounts.handovers SET status = 'taken_over', taken_over_at = now(), takeover_source = '
     checklist'
     WHERE order_id = p_order;
    INSERT INTO game_accounts.outbox (event_type, aggregate_id, payload)
    VALUES ('game_accounts.takeover.completed', p_order, jsonb_build_object('source', 'checklist', 'at', now
     ()));
    RETURN true;
  END IF;
  RETURN false;
END $$;

-- FR-GAM-15: the window-end job. Uses the statement clock, so that a job transaction started
-- just before the deadline still sees it pass. Completion time is the end of the window.
CREATE FUNCTION game_accounts.complete_if_due(p_order uuid) RETURNS timestamptz LANGUAGE plpgsql AS $$
DECLARE h game_accounts.handovers;
BEGIN
  SELECT * INTO h FROM game_accounts.handovers WHERE order_id = p_order FOR UPDATE;
  IF NOT FOUND OR h.status <> 'delivered' OR clock_timestamp() < h.takeover_due_at THEN RETURN NULL; END IF;
  UPDATE game_accounts.handovers SET status = 'taken_over', taken_over_at = h.takeover_due_at,
     takeover_source = 'window_end'
   WHERE order_id = p_order;
  INSERT INTO game_accounts.outbox (event_type, aggregate_id, payload)
  VALUES ('game_accounts.takeover.completed', p_order, jsonb_build_object('source', 'window_end', 'at', h.
     takeover_due_at));
  RETURN h.takeover_due_at;
END $$;
```

```
-- FR-GAM-14: "the credentials do not work", accepted only inside the window and only until the
-- buyer has confirmed changing the recovery e-mail. The service opens the dispute in the same
-- transaction (Unit 7).
CREATE FUNCTION game_accounts.report_invalid(p_order uuid, p_buyer uuid) RETURNS void LANGUAGE plpgsql AS $$
DECLARE h game_accounts.handovers;
BEGIN
  SELECT * INTO h FROM game_accounts.handovers WHERE order_id = p_order FOR UPDATE;
  IF NOT FOUND OR p_buyer <> h.buyer_id THEN
    RAISE EXCEPTION 'only␣the␣buyer␣can␣report' USING ERRCODE = 'insufficient_privilege';
  END IF;
  IF h.recovery_email_at IS NOT NULL THEN
    RAISE EXCEPTION 'the␣recovery␣e-mail␣was␣confirmed␣as␣changed;␣credentials_invalid␣is␣no␣longer␣accepted'
      USING ERRCODE = 'check_violation';
  END IF;
  IF h.status <> 'delivered' OR now() >= h.takeover_due_at THEN
    RAISE EXCEPTION 'a␣credentials_invalid␣report␣is␣accepted␣only␣during␣the␣takeover␣window' USING ERRCODE
     = 'check_violation';
  END IF;
  UPDATE game_accounts.handovers SET status = 'disputed', disputed_at = now() WHERE order_id = p_order;
  INSERT INTO game_accounts.outbox (event_type, aggregate_id, payload)
  VALUES ('game_accounts.credentials.reported_invalid', p_order, '{}');
END $$;

-- FR-GAM-12: one re-key during the window, before the recovery e-mail step; the window stops
-- until the seller seals again (re-delivery deadline DV-37).
CREATE FUNCTION game_accounts.request_rekey(p_order uuid, p_buyer uuid, p_key bytea, p_session uuid,
    p_redeliver interval)
RETURNS smallint LANGUAGE plpgsql AS $$
DECLARE h game_accounts.handovers;
BEGIN
  SELECT * INTO h FROM game_accounts.handovers WHERE order_id = p_order FOR UPDATE;
  IF NOT FOUND OR p_buyer <> h.buyer_id THEN
    RAISE EXCEPTION 'only␣the␣buyer␣can␣re-key' USING ERRCODE = 'insufficient_privilege';
  END IF;
  IF h.rekeys_used >= 1 THEN RAISE EXCEPTION 'the␣one␣re-key␣has␣been␣used' USING ERRCODE = 'check_violation'
    ; END IF;
  IF h.recovery_email_at IS NOT NULL THEN
    RAISE EXCEPTION 'the␣buyer␣already␣controls␣the␣recovery␣e-mail;␣no␣re-key␣is␣needed' USING ERRCODE = '
    check_violation';
  END IF;
  IF h.status <> 'delivered' OR now() >= h.takeover_due_at THEN
    RAISE EXCEPTION 'a␣re-key␣is␣possible␣only␣during␣the␣takeover␣window' USING ERRCODE = 'check_violation';
  END IF;
  INSERT INTO game_accounts.buyer_keys (order_id, version, public_key, session_id) VALUES (p_order, h.
    key_version + 1, p_key, p_session);
  UPDATE game_accounts.handovers
     SET status = 'rekey_requested', key_version = key_version + 1, rekeys_used = rekeys_used + 1,
         rekey_requested_at = now(), rekey_due_at = now() + p_redeliver, takeover_due_at = NULL
   WHERE order_id = p_order;
  INSERT INTO game_accounts.outbox (event_type, aggregate_id, payload)
  VALUES ('game_accounts.rekey.requested', p_order, jsonb_build_object('keyVersion', h.key_version + 1, '
    dueAt', now() + p_redeliver));
  RETURN h.key_version + 1;
END $$;

-- DV-37: the seller did not seal again in time; the case goes to a dispute (Unit 7)
CREATE FUNCTION game_accounts.rekey_overdue(p_order uuid) RETURNS boolean LANGUAGE plpgsql AS $$
BEGIN
  UPDATE game_accounts.handovers SET status = 'disputed', disputed_at = clock_timestamp(), rekey_due_at =
    NULL
   WHERE order_id = p_order AND status = 'rekey_requested' AND clock_timestamp() >= rekey_due_at;
  IF NOT FOUND THEN RETURN false; END IF;
  INSERT INTO game_accounts.outbox (event_type, aggregate_id, payload)
  VALUES ('game_accounts.rekey.overdue', p_order, '{}');
  RETURN true;
END $$;

-- a credentials_invalid dispute withdrawn by the buyer (FR-DSP-10): the takeover resumes; if its
-- window has already ended, the next job run completes it
CREATE FUNCTION game_accounts.dispute_withdrawn(p_order uuid) RETURNS void LANGUAGE plpgsql AS $$
BEGIN
  UPDATE game_accounts.handovers SET status = 'delivered', takeover_due_at = coalesce(takeover_due_at, now())
    , rekey_due_at = NULL
   WHERE order_id = p_order AND status = 'disputed';
  IF NOT FOUND THEN RAISE EXCEPTION 'handover␣is␣not␣disputed' USING ERRCODE = 'check_violation'; END IF;
```

```
END $$;

-- escrow refunded before the delivery event, or escrow terminal after it (from Unit 2 events):
-- the ciphertext is deleted (FR-GAM-19); after a sale the claim is released (FR-GAM-04)
CREATE FUNCTION game_accounts.finish(p_order uuid, p_before_delivery boolean) RETURNS void LANGUAGE plpgsql
     AS $$
DECLARE h game_accounts.handovers;
BEGIN
  SELECT * INTO h FROM game_accounts.handovers WHERE order_id = p_order FOR UPDATE;
  IF NOT FOUND OR h.status IN ('cancelled', 'closed') THEN RETURN; END IF;
  UPDATE game_accounts.sealed_credentials SET enc = NULL, ciphertext = NULL, deleted_at = now()
   WHERE order_id = p_order AND deleted_at IS NULL;
  IF p_before_delivery THEN
    UPDATE game_accounts.handovers SET status = 'cancelled', cancelled_at = now() WHERE order_id = p_order;
  ELSE
    UPDATE game_accounts.handovers SET status = 'closed', closed_at = now() WHERE order_id = p_order;
    UPDATE game_accounts.claims SET released_at = now(), release_reason = 'escrow_terminal'
     WHERE listing_id = h.listing_id AND released_at IS NULL;
  END IF;
END $$;

CREATE FUNCTION game_accounts.closed_without_ciphertext() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.status IN ('cancelled', 'closed') AND EXISTS (SELECT 1 FROM game_accounts.sealed_credentials
       WHERE order_id = NEW.order_id AND deleted_at IS NULL) THEN
    RAISE EXCEPTION 'FR-GAM-19: ciphertext must be deleted when the handover ends' USING ERRCODE = '
     check_violation';
  END IF;
  RETURN NULL;
END $$;
CREATE TRIGGER handover_ciphertext_deleted AFTER UPDATE OF status ON game_accounts.handovers
  FOR EACH ROW EXECUTE FUNCTION game_accounts.closed_without_ciphertext();

-- ====================================================================
-- Grants (SEC-INF-03)
-- ====================================================================
GRANT USAGE ON SCHEMA game_accounts TO tb_app;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA game_accounts TO tb_app;
REVOKE UPDATE ON game_accounts.access_log, game_accounts.confirmations, game_accounts.buyer_keys FROM tb_app;
```

C Browser Module

The reference module `sealed.js`, as tested. Unit 8 imports it unchanged.

```
// TrustBazaar - Unit 5 reference implementation of the sealed credential handover (browser side).
// Runs only in the buyer's and seller's browsers. The server never sees a private key or plaintext
// (SEC-GAM-01). HPKE (RFC 9180) base mode with DHKEM(P-256, HKDF-SHA256), HKDF-SHA256 and
// AES-256-GCM, through @hpke/core, which is built only on the Web Cryptography API (SEC-DAT-10,
// C-4: no construction of our own).
import { Aes256Gcm, CipherSuite, DhkemP256HkdfSha256, HkdfSha256 } from "@hpke/core";

const suite = new CipherSuite({ kem: new DhkemP256HkdfSha256(), kdf: new HkdfSha256(), aead: new Aes256Gcm()
    });
const INFO = new TextEncoder().encode("TrustBazaar/game-credentials/v1");          // DV-35
const BLOCK = 512, MAX = 4096;                                                      // DV-34
const aad = (orderId, keyVersion) => new TextEncoder().encode(`order:${orderId}|key:${keyVersion}`);

// FR-GAM-07: a P-256 key pair whose private key can never be exported
export async function generateBuyerKey() {
  const pair = await crypto.subtle.generateKey({ name: "ECDH", namedCurve: "P-256" }, false, ["deriveBits"]);
  const publicRaw = new Uint8Array(await crypto.subtle.exportKey("raw", pair.publicKey));   // 65 bytes, 0x04
    ...
  return { pair, publicRaw };
}

// the key pair is kept in IndexedDB of this browser only (structured clone keeps it non-extractable)
function db() {
  return new Promise((ok, fail) => {
    const r = indexedDB.open("tb-game-keys", 1);
    r.onupgradeneeded = () => r.result.createObjectStore("keys");
    r.onsuccess = () => ok(r.result); r.onerror = () => fail(r.error);
```

```
  });
}
export async function storeKey(orderId, version, pair) {
  const d = await db();
  await new Promise((ok, fail) => { const t = d.transaction("keys", "readwrite");
    t.objectStore("keys").put(pair, `${orderId}:${version}`); t.oncomplete = ok; t.onerror = () => fail(t.
    error); });
}
export async function loadKey(orderId, version) {
  const d = await db();
  return new Promise((ok, fail) => { const r = d.transaction("keys").objectStore("keys").get(`${orderId}:${
    version}`);
    r.onsuccess = () => ok(r.result || null); r.onerror = () => fail(r.error); });
}

// FR-GAM-08: first 80 bits of SHA-256 over the raw public key, five groups of four hex digits (DV-36)
export async function fingerprint(publicRaw) {
  const h = new Uint8Array(await crypto.subtle.digest("SHA-256", publicRaw));
  const hex = [...h.slice(0, 10)].map((b) => b.toString(16).padStart(2, "0")).join("");
  return hex.match(/.{4}/g).join("-");
}

// DV-34: 2-byte length, the JSON, zero padding to a multiple of 512 bytes; at most 4096 bytes
function pad(obj) {
  const body = new TextEncoder().encode(JSON.stringify(obj));
  const total = Math.max(BLOCK, Math.ceil((body.length + 2) / BLOCK) * BLOCK);
  if (total > MAX) throw new Error("credentials too long");
  const out = new Uint8Array(total);
  out[0] = body.length >> 8; out[1] = body.length & 255; out.set(body, 2);
  return out;
}
function unpad(bytes) {
  const n = (bytes[0] << 8) | bytes[1];
  return JSON.parse(new TextDecoder().decode(bytes.slice(2, 2 + n)));
}

// FR-GAM-09: the seller's browser seals the credentials for the buyer's current key
export async function seal(publicRaw, orderId, keyVersion, credentials) {
  const recipientPublicKey = await suite.kem.deserializePublicKey(publicRaw);
  const sender = await suite.createSenderContext({ recipientPublicKey, info: INFO });
  const ciphertext = new Uint8Array(await sender.seal(pad(credentials), aad(orderId, keyVersion)));
  return { enc: new Uint8Array(sender.enc), ciphertext };
}

// FR-GAM-11, FR-GAM-13: the buyer's browser opens them with the stored, non-extractable key
export async function open(pair, orderId, keyVersion, enc, ciphertext) {
  const recipient = await suite.createRecipientContext({ recipientKey: pair, enc, info: INFO });
  return unpad(new Uint8Array(await recipient.open(ciphertext, aad(orderId, keyVersion))));
}
```